SYSTEM SPECIFICATION DOCUMENT

RISE-Phish: Software Requirements Specification (SRS)

IEEE Std 830-1998 Compliant Specification for Cross-Representation Discrepancy-Aware Phishing Detection

--- SOFTWARE REQUIREMENTS SPECIFICATION (SRS) ---

Submitted By:	[Student Name 1] (Roll No: [Roll No 1]) [Student Name 2] (Roll No: [Roll No 2]) [Student Name 3] (Roll No: [Roll No 3])
Project Guide / Supervisor:	[Project Guide Name] [Guide Designation & Department]
Department:	Department of Artificial Intelligence & Machine Learning / Computer Science
Institution:	[Institution / College / University Name]
Academic Year:	[Academic Year: 202X - 202Y]

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document details the complete functional and non-functional requirements for the RISE-Phish platform. RISE-Phish is a final-year B.Tech engineering capstone system engineered for discrepancy-aware phishing detection using multiple webpage representations, optical character recognition (OCR), Unicode confusable analysis, lightweight small language models (SLM), and browser extension integration. This document conforms to the IEEE Std 830-1998 format.

1.2 Document Conventions

Every requirement in this specification is assigned a unique alphanumeric identifier: FR-[Number] denotes Functional Requirements, and NFR-[Number] denotes Non-Functional Requirements. Priorities are classified using MoSCoW definitions: MUST (essential for core release), SHOULD (highly desirable capability), and COULD (optional enhancement).

1.3 Intended Audience and Reading Suggestions

This document is intended for project evaluators, software architects, cybersecurity researchers, and developers. Readers interested in architectural logic should focus on Section 2 and Section 4; security analysts should review Section 5.3.

1.4 Product Scope

RISE-Phish provides automated, real-time detection of deceptive web destinations, credential harvesting pages, and phishing messages. The system simultaneously captures five representation layers (DOM R1, Normalized R2, Visual OCR R3, Semantic Intent R4, URL Lexical R5), computes their cross-representation divergence, detects Visual-Normalization Convergence (VNC), fuses multi-modal threat signals, and generates dual-audience explanations for everyday users and SOC analysts.

1.5 References

- **[1]** IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998).
- **[2]** Unicode Technical Standard #39: Unicode Security Mechanisms (The Unicode Consortium, 2024).
- **[3]** FastAPI Documentation and OpenAPI Specification 3.1.0.
- **[4]** Google Chrome Manifest V3 Security Architecture and Extension Standards.

2. OVERALL DESCRIPTION

2.1 Product Perspective

RISE-Phish is a hybrid endpoint-assisted and gateway-capable security system. It operates as an autonomous REST API service (FastAPI), coupled with an interactive web dashboard (React 19) and a

lightweight endpoint agent (Chrome/Edge Manifest V3 extension). It interfaces with sandboxed Chromium instances and local Tesseract OCR to perform multi-modal webpage deconstruction.

2.2 Product Functions Overview

- **1. Web Crawling & Acquisition:** Ingests target URLs, launches sandboxed Chromium, renders dynamic DOM, captures high-resolution screenshots, and tracks HTTP redirects.
- **2. Visual OCR Extraction:** Extracts optical text from rendered screen pixels using Tesseract OCR, representing exact visual victim perception.
- **3. Representation Compilation:** Generates representations R1 through R5, standardizing Unicode via NFKC and mapping traceable confusable glyphs.
- **4. Discrepancy Engine (CRCE):** Calculates Normalized Edit Distance and Token Jaccard Divergence across all pairs, identifying VNC signatures.
- **5. SLM Semantic Reasoning:** Executes lightweight SLM inference (0.5B-1.5B) for brand identity resolution and context-aware phishing threat justification.
- **6. Deterministic Fusion:** Combines weighted evidence (CRCE, URL, Unicode, SLM) with non-linear boost multipliers to output calibrated threat scores.
- **7. Dual Explainability:** Produces plain-language action guidance for users and structured forensic workbenches for SOC tier-1/tier-2 analysts.
- **8. Browser Extension:** Monitors active web navigation, presents dynamic warning banners, and inspects selected message/email text.

2.3 User Classes and Characteristics

User Class	Description and Operational Needs	Primary Interface
Everyday End-User	Non-technical web user requiring clear, jargon-free security guidance, immediate blocking of malicious sites, and low navigation latency.	Desktop Chrome/Edge Browser
SOC Analyst (Tier-1 / Tier-2)	Security professional investigating escalated incidents, analyzing exact Unicode codepoints, inspecting discrepancy matrices, and submitting feedback.	Web Dashboard Forensic View
System Administrator	Infrastructure engineer managing deployment, configuring CORS, setting API thresholds, and monitoring system resource usage.	CLI, Docker, .env Configuration
Academic Evaluator / Researcher	Auditor inspecting reproducible experimental benchmarks, ablation studies, and mathematical formulation validity.	Pytest Suite & Evaluation Reports

2.4 Operating Environment

The software operates in the following verified environment:

- **Operating System:** Windows 11 (64-bit) or Ubuntu 22.04 LTS.
- **Software Runtimes:** Python 3.12.5 (Backend), Node.js 18.x+ (Frontend build).
- **System Dependencies:** Playwright 1.48+ (Chromium headless), Tesseract OCR 5.4+.
- **Hardware Baseline:** Target hardware: NVIDIA GeForce RTX 3050 (4 GB VRAM), 16 GB RAM, multi-core CPU. Fallback supported on pure CPU.

2.5 Design and Implementation Constraints

- **Latency Constraint:** The entire pipeline must execute within 15 seconds maximum hard timeout per URL to avoid user navigation blockage.
- **Memory Constraint:** Local SLM execution must fit within 4.0 GB VRAM using 4-bit or 8-bit quantization, with deterministic rule-based fallback when SLM is disabled.
- **Security Constraint:** Playwright crawler must enforce loopback/private IP blocking to mitigate Server-Side Request Forgery (SSRF).
- **Extension Constraint:** Browser extension must comply with Manifest V3 restrictions (no remote code execution, declarativeNetRequest, service worker lifecycles).

3. EXTERNAL INTERFACE REQUIREMENTS

3.1 User Interfaces

UI-1: Web Dashboard: Built with React 19 and Tailwind CSS. Features URL submission bar, threat meter (0-100), operational verdict badge (SAFE, INVESTIGATE, PHISHING), visual representation diffing inspector, and dual explanation cards.

UI-2: In-Page Alert Banner: Injected by extension content script. Displays full-width banner at top of webpage with color-coded severity: Green (Safe), Yellow (Investigate), Red (Phishing).

UI-3: Extension Popup: Displays current tab scan status, representation tokens, email text snippet scanner, and backend connectivity indicator.

3.2 Hardware Interfaces

The system interfaces with NVIDIA GPUs via CUDA 12.x for accelerated SLM tensor inference. If CUDA is unavailable, the system transparently falls back to CPU multi-threaded execution.

3.3 Software Interfaces

- **Tesseract OCR Engine:** Invoked via subprocess or pytesseract wrapper to extract textual strings from rendered viewport PNGs.
- **Playwright Engine:** Controlled via Python async API to navigate URLs, bypass bot mitigations, capture DOM, and screenshot canvases.
- **Database Interface:** Manages relational persistence in SQLite (database.db) using SQLAlchemy ORM.

3.4 Communications Interfaces

All communications between frontend, extension, and backend occur over HTTP/1.1 REST using JSON payloads. Key REST endpoints include: POST /analyze, POST /extension/analyze, POST /extension/check, GET /history, and POST /feedback.

4. SPECIFIC FUNCTIONAL REQUIREMENTS

Req ID	Requirement Title	Functional Specification Description	Priority
FR-1	URL Ingestion & Sandboxed Crawling	The system MUST accept target URLs, launch an isolated Chromium process, enforce a 15-second timeout, capture the raw HTML DOM, and extract network redirect chains.	MUST
FR-2	Visual Viewport & OCR Ground Truth	The system MUST capture a 1280x720 PNG viewport screenshot and execute Tesseract OCR to produce visual representation R3 reflecting exact optical text.	MUST
FR-3	Multi-Representation Suite (R1-R5)	The system MUST extract raw DOM text (R1), compute canonical NFKC text with confusable resolution (R2), store visual OCR (R3), extract form actions (R4), and decompose URL tokens (R5).	MUST
FR-4	Unicode Anomaly & Homoglyph Detection	The system MUST calculate character-level entropy, identify zero-width steganographic characters (\u200B-\u200D), and detect mixed-script confusable substitutions.	MUST
FR-5	Cross-Representation Discrepancy (CRCE)	The system MUST compute Normalized Edit Distance and Token Jaccard Divergence across all representation pairs and mathematically identify the VNC signature.	MUST
FR-6	Brand Semantic Recovery	The system MUST evaluate normalized and visual representations against a canonical brand identity dictionary to detect spoofed brand targets.	MUST
FR-7	SLM Phishing Reasoning	The system MUST invoke small language model inference (or rule-based fallback) to generate multi-modal threat justification.	SHOULD
FR-8	Deterministic Evidence Fusion	The system MUST fuse CRCE (35%), URL (25%), Unicode (25%), and SLM (15%) scores, applying deterministic non-linear boost multipliers for high-risk co-occurrences.	MUST
FR-9	Operational Decision & Postures	The system MUST categorize composite threat scores into SAFE (0-39), INVESTIGATE (40-69), and PHISHING (70-100) and assign automated postures (ALLOW, WARN, BLOCK).	MUST
FR-10	Dual-Audience Explainability	The system MUST generate plain-language explanations for end-users and structured forensic JSON records for SOC analysts.	MUST

Req ID	Requirement Title	Functional Specification Description	Priority
FR-11	Browser Extension Tab Protection	The extension MUST monitor tab navigation, query the backend API, and inject security banners into suspicious active pages.	MUST
FR-12	Extension Message & Email Inspection	The extension MUST allow users to highlight webmail text or input message snippets to evaluate phishing risk prior to clicking embedded links.	MUST
FR-13	Trust List Whitelist Guardrail	The system MUST maintain an authoritative trust list of high-reputation domains that bypass heavy crawls, clamping scores to 0.0 unless spoofing is detected.	MUST
FR-14	Audit Logging & Analyst Feedback	The system MUST persist all scan records and allow analysts to submit ground-truth feedback labels for continuous benchmarking.	MUST

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance Requirements

NFR-1.1: End-to-end analysis latency on local GPU hardware (RTX 3050) SHALL NOT exceed 650 milliseconds for cached or standard-sized pages.

NFR-1.2: CPU fallback execution latency SHALL NOT exceed 1000 milliseconds.

NFR-1.3: C++ accelerated string metric computation (RapidFuzz) SHALL complete in under 10 milliseconds for representations up to 100,000 characters.

5.2 Security Requirements

NFR-2.1: The Playwright crawler MUST block internal network navigation, loopback IP addresses (127.0.0.1, ::1), and cloud metadata endpoints (169.254.169.254) to prevent Server-Side Request Forgery (SSRF).

NFR-2.2: The browser extension MUST NOT inject external script tags or execute unvetted code, adhering to Chrome Manifest V3 Content Security Policy (CSP).

NFR-2.3: API keys and sensitive environment variables MUST NOT be logged or exposed in client responses.

5.3 Reliability and Availability

NFR-3.1: If Playwright fails due to network drop or target refusal, the system MUST gracefully fall back to syntactic URL analysis and return an informative status code.

NFR-3.2: If Tesseract OCR fails or yields empty strings, the system MUST proceed using DOM and Normalized representations without crashing.

NFR-3.3: If the SLM model fails to load or exceeds memory bounds, the deterministic rule engine MUST activate automatically.

5.4 Software Quality Attributes

- **Modularity:** All core algorithms are decoupled into standalone Python modules with no cyclic dependencies.
- **Testability:** The automated test suite achieves 100% pass rate across 177 unit and integration tests.
- **Portability:** Docker and Docker Compose configurations enable containerized deployment across Windows, Linux, and macOS.

6. REQUIREMENTS TRACEABILITY MATRIX

Table 6.1 maps each functional requirement to its implementing backend module and corresponding automated pytest validation suite:

Requirement ID & Title	Implementing Code Module	Verifying Test Suite
FR-1: Sandboxed Crawling	backend/app/core/crawler.py	tests/test_crawler.py (12 tests)
FR-2: Visual OCR	backend/app/core/vision.py	tests/test_vision.py (8 tests)
FR-3: Multi-Rep Suite	backend/app/core/rep_builder.py	tests/test_rep_builder.py (14 tests)
FR-4: Unicode Analysis	backend/app/core/unicode_analyzer.py	tests/test_unicode_analyzer.py (18 tests)
FR-5: CRCE Discrepancies	backend/app/core/crce.py	tests/test_crce.py (16 tests)
FR-6: Brand Recovery	backend/app/slm/recovery.py	tests/test_recovery.py (11 tests)
FR-7: SLM Reasoning	backend/app/slm/reasoning.py	tests/test_reasoning.py (14 tests)
FR-8: Evidence Fusion	backend/app/core/fusion.py	tests/test_fusion.py (15 tests)
FR-9: Decision Engine	backend/app/core/decision.py	tests/test_decision.py (10 tests)
FR-10: Dual Explainability	backend/app/core/explainability.py	tests/test_explainability.py (12 tests)
FR-11: Extension Tab Scan	browser-extension/src/content.js	tests/test_extension_api.py (8 tests)
FR-12: Extension Message Scan	browser-extension/src/popup.js	tests/test_extension_api.py (8 tests)
FR-13: Trust List Override	backend/app/core/decision.py	tests/test_decision.py (5 tests)
FR-14: Audit DB & Feedback	backend/app/db/database.py & api/routes.py	tests/test_feedback.py (8 tests)